

Software Requirements Specification

for

Clinic Appointment and Treatment Management System (CATMS)

Phase 1: Appointment Booking, Treatment Recording, and Billing Database Design

Version 1.0

Prepared for: MedSync Clinics

Prepared by:

CS3048: Database Management Systems — Course Project

Date: 2026.07.28

Table of Contents

Table of Contents	1
Revision History	2
1. Introduction	2
1.1 Purpose	3
1.2 Document Conventions	3
1.3 Intended Audience and Reading Suggestions	3
1.4 Product Scope	3
1.5 References	4
2. Overall Description	4
2.1 Product Perspective	5
System Architecture Diagram	5
2.2 Product Functions	5
2.3 User Classes and Characteristics	6
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 User Documentation	7
2.7 Assumptions and Dependencies	7
3. External Interface Requirements	7
3.1 User Interfaces	8
3.2 Hardware Interfaces	8
3.3 Software Interfaces	8
3.4 Communications Interfaces	9
4. System Features	9
4.1 Branch and Staff Management	10
4.1.1 Description and Priority	10
4.1.2 Stimulus/Response Sequences	10
4.1.3 Functional Requirements	10
4.2 Patient and Insurance Management	10
4.2.1 Description and Priority	11
4.2.2 Stimulus/Response Sequences	11
4.2.3 Functional Requirements	11
4.3 Appointment Booking and Scheduling	11
4.3.1 Description and Priority	11

4.3.2 Stimulus/Response Sequences	11
4.3.3 Functional Requirements.....	12
4.4 Treatment and Consultation Recording	12
4.4.1 Description and Priority	12
4.4.2 Stimulus/Response Sequences	12
4.4.3 Functional Requirements.....	13
4.5 Billing, Payments, and Insurance Claims	13
4.5.1 Description and Priority	13
4.5.2 Stimulus/Response Sequences	13
4.5.3 Functional Requirements.....	14
4.6 Management Reporting.....	14
4.6.1 Description and Priority	14
4.6.2 Stimulus/Response Sequences	14
4.6.3 Functional Requirements.....	15
4.7 QA / Testing User Interface	15
4.7.1 Description and Priority	15
4.7.2 Stimulus/Response Sequences	15
4.7.3 Functional Requirements.....	15
5. Other Nonfunctional Requirements	16
5.1 Performance Requirements.....	17
5.2 Safety Requirements.....	17
5.3 Security Requirements.....	17
5.4 Software Quality Attributes	17
5.5 Business Rules.....	18
6. Other Requirements	18
6.1 Database / Transaction Requirements	19
6.2 Data Modelling and Normalization Requirements	19
6.3 Indexing Requirements.....	19
6.4 Dummy Data Requirements	20
6.5 Internationalisation and Localisation Requirements	20
6.6 Logical Structure of the Data (Data Dictionary).....	20
Module A — Branch & Staff Management.....	20
Table: Branch	21
Table: Appointment_Status_Log	21
Module D — Treatment & Billing	21

Table: Treatment_Catalogue	21
Table: Invoice	21
Module E — Payments, Claims & Reporting	22
Table: Payment	22
6.7 Requirements Traceability Matrix	22
Appendix A: Glossary	23
Appendix B: Analysis Models	24
B.3 Use Case Diagram	25
B.1 Conceptual Modules	26
B.2 Key Relationships and Cardinality	27
Appendix C: To Be Determined List	27

Revision History

Name	Date	Reason For Changes	Version
Group 6	2026.07.28	Initial draft of SRS for CATMS Phase 1, based on the client brief and the database design plan.	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for Phase 1 of the Clinic Appointment and Treatment Management System (CATMS), version 1.0. CATMS is being developed for MedSync, a medium-scale multi-specialty clinic operating three branches (Colombo, Kandy, and Galle), to replace its current mixture of paper records and Excel sheets.

Phase 1 focuses specifically on digitising three processes appointment booking, treatment recording, and billing (including insurance claims) with an emphasis on database correctness. A quality assurance (QA) user interface is also specified, but only to the extent needed to exercise and demonstrate the database's behaviour; the UI itself is not the primary graded deliverable.

1.2 Document Conventions

Each functional requirement is uniquely identified with the tag REQ-<n> (e.g. REQ-14), and requirements are grouped under the system feature they belong to (Section 4). Unless stated otherwise, all functional requirements in each feature share the priority stated for that feature; individual requirements are not separately re-prioritised. The keyword “shall” indicate a mandatory requirement; “should” indicates a recommended, non-mandatory characteristic. Table and column names that correspond directly to the underlying database schema are written in monospace-style capitalisation (e.g. Appointment, Treatment_Catalogue) to distinguish them from ordinary prose.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Database designers and developers implementing the schema, triggers, procedures, and views described here.
- The course instructor / project evaluator, to assess problem definition, objectives, and completeness of requirement analysis.
- QA testers who will exercise the system against the functional requirements in Section 4.
- MedSync clinic management, as the eventual business stakeholder for the reporting requirements in Section 4.6.

Readers primarily interested in the business problem and objectives should read Sections 1 and 2. Readers implementing or reviewing the database should focus on Sections 2.5, 4, 5, and 6. Readers verifying test coverage should focus on Section 4, where every feature lists its stimulus/response behaviour and numbered requirements.

1.4 Product Scope

CATMS Phase 1 is a database centred system that: (a) maintains a single, clinic-wide record for every patient, doctor, and branch; (b) guarantees that a doctor is never double-booked; (c) records treatments and consultation notes only once an appointment is completed; (d) automatically generates invoices from completed treatments and tracks full or partial payments and outstanding dues; (e) supports partial or full insurance reimbursement per treatment; and (f) produces the five management reports listed in Section 4.6. All business rules that guarantee correctness (no double-booking, no treatment

before completion, no overpayment, etc.) are enforced inside the database itself through primary/foreign keys, CHECK constraints, triggers, and stored procedures rather than being trusted to the application layer, so that the rules hold even if the QA UI is bypassed.

Explicitly out of scope for Phase 1: a production-grade, patient-facing user interface; multi-currency support (all amounts are in LKR); integration with external insurance-provider systems (claims are tracked internally, not transmitted electronically); and branch-specific treatment pricing (treatment prices are clinic-wide in Phase 1 see Section 2.7).

1.5 References

- MedSync Clinic — “Project 1: Clinic Appointment and Treatment Management System”, client brief and task description.
- CATMS Database Engineering Specification, v1.0 internal design document covering the conceptual model, entity/attribute definitions, functional dependencies and normalization proof, DDL, indexing strategy, triggers/procedures, reporting views, security model, and UI wireframes referenced throughout this SRS.
- IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998), used as the structural template for this document.

2. Overall Description

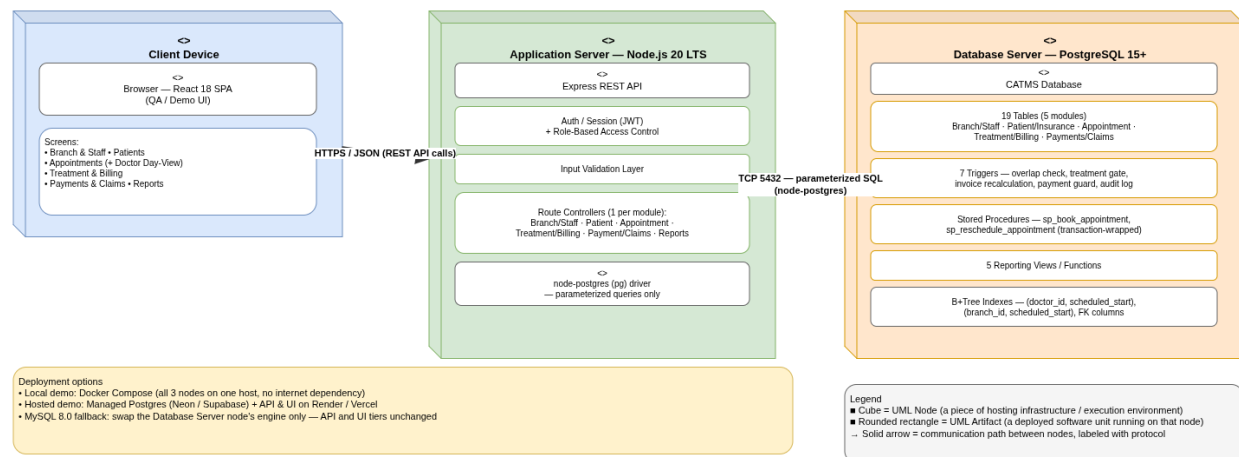
2.1 Product Perspective

CATMS is a new, self-contained system replacing MedSync's existing paper-and-spreadsheet process; it is not a follow-on to an existing software product. The system follows a three-tier architecture:

- Tier 1 — Presentation: a browser-based QA user interface (React) used to exercise the system's features.
- Tier 2 — Business logic: a thin REST API (Node.js/Express) that performs authentication, input validation, and dispatches to the database's stored procedures and views.
- Tier 3 — Data: a relational database (PostgreSQL) holding all tables, triggers, stored procedures, functions, and reporting views. This tier owns every business rule that must always hold true, so that Tier 2 cannot be bypassed to violate them.

System Architecture Diagram

CATMS — System Architecture (UML Deployment Diagram, 3-Tier)



This layering gives physical and logical data independence: the UI and reports interact only with the logical schema (tables and views) in Tier 3, so storage-level changes (new indexes, partitioning) can be made without touching application code.

2.2 Product Functions

At a high level, CATMS Phase 1 must let authorised users:

- Manage branches, staff, doctors, and specialties.
- Register and search for patient's clinic-wide and manage their insurance policies.
- Book, reschedule, cancel, and walk-in appointments with automatic doctor-overlap prevention.
- Record treatments and consultation note against completed appointments only.
- Generate invoices automatically, accept full or partial payments, and manage insurance claims.
- Produce the five required management reports.

- Exercise all the above through a minimal, role based QA interface.

2.3 User Classes and Characteristics

User Class	Characteristics
Receptionist	Front-desk staff at a branch. Registers patients, books/reschedules/cancels appointments, creates walk-ins. Frequent, routine use; no access to billing internals or reports.
Doctor / Clinician	Records treatments, consultation notes, and diagnoses against completed appointments. Views (but does not edit) invoices. Moderate frequency of use specialised to clinical tasks.
Branch Manager	An Employee with role = Manager for a specific branch. Reviews branch-level operational data such as the daily appointment summary.
Admin / Finance Staff	Manages payments, insurance claims, and treatment catalogue pricing; the only user class with access to all five management reports and full billing data.
QA Tester	Uses the role-based QA UI to verify that every business rule (Sections 4 and 5) is correctly enforced by the database, including deliberately triggering rejected operations.

The most important user classes to satisfy are Receptionist, Doctor/Clinician, and Admin/Finance Staff, since these three classes cover the entire operational and financial workflow the database must guarantee. The QA Tester class is important for verification but is not a production stakeholder.

2.4 Operating Environment

- Database: PostgreSQL 15 or later (MySQL 8.0).
- Backend: Node.js 20 with Express, communicating with the database via the node Postgres (pg) driver using exclusively parameterized queries.
- Frontend: a modern evergreen web browser running the React 18 QA UI, served over HTTPS.
- Hosting: either a managed PostgreSQL instance with the API/UI deployed to a cloud host, or a fully local Docker Compose stack for offline demonstration.

2.5 Design and Implementation Constraints

- All rules required for correctness (doctor-overlap prevention, treatment gating, invoice recalculation, payment caps) must be implemented at the database level (constraints, triggers, stored procedures), not solely in application code, per the client brief's requirement to identify "procedures, functions, and triggers ... to guarantee ACID properties."
- Every primary key and foreign key relationship must be explicitly defined with an appropriate ON DELETE policy (RESTRICT, CASCADE, or SET NULL) chosen per relationship, not left to the database default.
- The schema must be normalized to at least Third Normal Form (3NF) / Boyce-Codd Normal Form (BCNF), with any deliberate denormalization (e.g. storing the treatment price at the time of billing) explicitly documented and justified.

- Every query issued by the application layer must be parameterized; string-concatenated SQL is prohibited (SQL-injection prevention, Section 5.3).
- All monetary values are stored as DECIMAL (10,2) in Sri Lankan Rupees (LKR); no multi-currency support is required in Phase 1.
- If the target DBMS is MySQL instead of PostgreSQL, the doctor-overlap rule must be enforced purely through a trigger (PostgreSQL's EXCLUDE constraint with the btree_gist extension is unavailable in MySQL).

2.6 User Documentation

The following documentation will be delivered alongside the software: (a) this SRS; (b) the CATMS Database Engineering Specification (schema DDL, ER diagram, trigger/procedure listings); (c) a short QA user guide describing how to log in with each role and exercise each screen listed in Section 3.1; and (d) inline database comments on non-obvious constraints and triggers.

2.7 Assumptions and Dependencies

Area	Assumption
Slot granularity	Appointments are booked in 15-minute increments; default consultation length is 30 minutes unless a treatment defines its own duration.
Consultation billing	The consultation itself is a row in the Treatment Catalogue, so it flows through the same billing/insurance pipeline as any other treatment rather than being a special case.
Treatment pricing	Prices are clinic-wide (branch-independent) for Phase 1; branch-specific pricing is deferred to a future phase.
Insurance terms	Coverage is defined per policy, per treatment, as a percentage plus an optional maximum claimable amount. A patient may hold policies from more than one provider.
Doctor specialties	Assigned at onboarding and treated as a fixed set for Phase 1; no history of specialty changes is tracked.
Walk-ins	Modelled as ordinary appointments with source = Walk-in, created by a staff member at the moment the patient arrives, rather than as a separate entity.
Cancellations	Soft state only: a cancelled appointment's row is retained (status = Cancelled) so it still appears correctly in the daily summary report.
Currency	All amounts are in LKR, stored as DECIMAL (10,2).
Branch manager	A manager is simply an Employee with role = Manager; a branch's manager reference may briefly be null immediately after the branch is created, until the first manager is hired/seeded.

3. External Interface Requirements

3.1 User Interfaces

The QA user interface is a single-page web application with a role-based login (Reception, Clinician, or Admin) followed by a dashboard with module tabs: Branch & Staff, Patients, Appointments, Treatment & Billing, Payments & Claims, and Reports.

Screen	Purpose
Branch & Staff	List branches; add employees; assign doctor specialties.
Patient Registration / Search	Register a patient at any branch; search across all branches by NIC, name, or phone.
Book Appointment	Select patient, branch, doctor, and time slot; create booked or walk-in appointments.
Doctor Day View	Timeline of a doctor's booked slots for a given day; used to demonstrate overlap rejection.
Reschedule / Cancel	Change an existing appointment's time or cancel it with a reason.
Record Treatment	Enabled only once an appointment's status is Completed; records treatments and consultation notes.
Invoice View	Read-only display of subtotal, insurance-covered amount, payable amount, amount paid, and status.
Post Payment	Record a full or partial payment against an invoice and show the resulting balance.
Insurance Claims	Submit or update an insurance claim against a policy and invoice.
Reports (x5)	One tab per required report, with a date-range filter where relevant.

Every screen shows the exact rejection message raised by the database when a business rule is violated, rather than a generic error message, so that the rule being enforced is visible to the user. Fields disabled by a business rule (e.g. "Record Treatment" before an appointment is Completed) are visibly greyed out.

3.2 Hardware Interfaces

CATMS Phase 1 has no direct hardware interfaces. The QA UI runs in a standard web browser on any device (desktop, laptop, or tablet) with network access to the API server; no specialised medical devices or peripherals are integrated in this phase.

3.3 Software Interfaces

Component	Interface
PostgreSQL 15+ (or MySQL 8.0 fallback)	The system of record for all data. Accessed exclusively through parameterized SQL, stored procedures, and views never through direct, unparameterized queries.
Node.js 20 / Express REST API	Exposes one route per stored procedure or reporting view; communicates with the database using the node-postgres (pg) driver.
React 18 QA UI	Communicates with the API exclusively over HTTPS using JSON payloads.

3.4 Communications Interfaces

All communication between the browser-based UI and the API is over HTTPS, using JSON request and response bodies. No email, SMS, or other external messaging interface is required for Phase 1. Authentication uses a session token or JSON Web Token (JWT), transmitted over HTTPS, with the underlying employee role stored in the database.

4. System Features

4.1 Branch and Staff Management

4.1.1 Description and Priority

Enables the clinic to register its branches (Colombo, Kandy, Galle) and manage the medical and non-medical staff attached to each branch, including doctors and their specialties. This feature underpins every other feature, since appointments, invoices, and reports are all scoped by branch and staff member.

Priority: High

4.1.2 Stimulus/Response Sequences

- Admin/Finance staff member adds a new branch → system stores the branch record and allows it to be selected wherever a branch is required (patient registration, appointment booking, reports).
- Admin/Finance staff member registers a new employee and assigns a role → system stores the employee record; if the role is “Doctor”, the system additionally requires a license number and permits one or more specialties to be attached.
- Admin/Finance staff member assigns an employee as branch manager → system updates the branch’s manager reference.

4.1.3 Functional Requirements

ID	Requirement
REQ-1	The system shall allow authorised staff to create, view, and update Branch records containing name, address, city, and contact phone number.
REQ-2	The system shall allow exactly one Employee to be designated as the manager of a Branch. A branch may temporarily have no manager assigned (e.g. immediately after seeding), but a manager, once assigned, must reference an existing, active employee.
REQ-3	The system shall allow registration of an Employee with full name, National Identity Card (NIC) number, gender, date of birth, role (Manager, Doctor, Nurse, Receptionist, Admin, Other), contact details, and hire date, and shall attach every employee to exactly one home branch. The NIC number shall be unique across the whole clinic.
REQ-4	The system shall allow an Employee whose role is “Doctor” to be additionally registered with a unique medical license number, a default consultation fee, and years of experience.
REQ-5	The system shall allow a Doctor to be associated with one or more Specialties (e.g. General Medicine, ENT, Paediatrics) and shall prevent the same specialty being assigned twice to the same doctor.
REQ-6	The system shall support deactivating (soft-deleting) an Employee record without physically deleting it, so that historical appointments and invoices referencing that employee remain intact.

4.2 Patient and Insurance Management

4.2.1 Description and Priority

Allows a patient to be registered once at any branch and be visible and treatable from any other branch, and allows the patient's insurance policies and coverage terms to be recorded for use during billing.

Priority: High

4.2.2 Stimulus/Response Sequences

- Receptionist registers a new patient at Branch A → the patient record becomes immediately visible and bookable from Branch B and Branch C.
- Receptionist searches for a patient by NIC/passport, name, or phone → system returns matching patients clinic-wide, regardless of registering branch.
- Staff member records an insurance policy for a patient → system stores policy details and, for each covered treatment, the coverage percentage and (optionally) a maximum claimable amount.

4.2.3 Functional Requirements

ID	Requirement
REQ-7	The system shall allow a patient to be registered at any branch and shall make that patient's record accessible for booking and treatment at every branch thereafter.
REQ-8	The system shall enforce uniqueness of a patient's NIC or passport number across the entire clinic so that a patient is never registered twice under two different records.
REQ-9	The system shall store a patient's personal details (full name, date of birth, gender, address, phone, email, blood group) and one or more emergency contacts (name, relationship, phone).
REQ-10	The system shall allow a patient to hold one or more Insurance Policies, each linked to an Insurance Provider, with a policy number, validity period, and status (Active, Expired, Suspended).
REQ-11	The system shall allow each Insurance Policy to define, per treatment in the catalogue, a coverage percentage (0–100%) and an optional maximum claimable amount.
REQ-12	The system shall allow a patient to simultaneously hold active policies from more than one insurance provider.

4.3 Appointment Booking and Scheduling

4.3.1 Description and Priority

Covers booking, rescheduling, cancelling, and walk-in creation of appointments between a patient and a doctor, with the core business rule that a doctor can never be double-booked for overlapping time slots.

Priority: High

4.3.2 Stimulus/Response Sequences

- Receptionist books an appointment for a patient with an available doctor → system creates the appointment with status “Scheduled”.
- Receptionist attempts to book a doctor for a time slot that overlaps one of the doctor’s existing (non-cancelled) appointments → system rejects the request and reports the conflict.
- Staff member creates a walk-in appointment for a patient who arrives without a prior booking → system creates the appointment immediately, tagged as a walk-in.
- Receptionist reschedules an existing appointment to a new date/time → system re-validates the new slot for overlap and records the change in the appointment history.
- Staff member marks an appointment as Completed or Cancelled → system records the transition in an audit log together with a timestamp and the staff member who made the change.

4.3.3 Functional Requirements

ID	Requirement
REQ-13	The system shall allow an appointment to be created between exactly one patient and one doctor at a specific branch, for a defined start and end time.
REQ-14	The system shall prevent a doctor from being booked into two appointments whose time ranges overlap, regardless of which branch or which staff member created either appointment. This rule shall be enforced at the database level so that it cannot be bypassed by any client application.
REQ-15	The system shall track the status of every appointment as one of: Scheduled, Completed, or Cancelled, and shall record every status transition together with a timestamp and the user who made the change.
REQ-16	The system shall allow an existing appointment to be rescheduled to a new date/time, re-applying the doctor-overlap rule (REQ-14) to the new slot, and shall retain a history record of the old and new time, the reason, and the staff member responsible.
REQ-17	The system shall allow a staff member to create a walk-in appointment directly, without a prior booking, following the same overlap and status rules as a booked appointment.
REQ-18	The system shall allow cancellation of a Scheduled appointment; a cancelled appointment shall be retained (not deleted) so that it is still counted correctly in the branch-wise daily summary report.

4.4 Treatment and Consultation Recording

4.4.1 Description and Priority

Covers recording of the treatments a doctor provides during a completed appointment, against a clinic-wide treatment catalogue, together with consultation notes.

Priority: High

4.4.2 Stimulus/Response Sequences

- Doctor attempts to record a treatment against an appointment that is still “Scheduled” → system rejects the request until the appointment is marked Completed.
- Doctor marks an appointment as Completed and then records one or more treatments (e.g. Consultation, X-Ray, ECG) and consultation notes → system stores each treatment with the price that applied at that time.
- Admin/Finance staff member adds a new treatment (service code, name, category, price) to the catalogue, or retires an existing one → the catalogue reflects the change without altering historical billing records.

4.4.3 Functional Requirements

ID	Requirement
REQ-19	The system shall only allow treatments and consultation notes to be recorded against an appointment whose status is Completed, and shall reject any attempt to record them against a Scheduled or Cancelled appointment.
REQ-20	The system shall allow one or more treatments to be selected from a pre-defined Treatment Catalogue and associated with a completed appointment, each with a quantity.
REQ-21	The system shall allow exactly one set of consultation notes/diagnosis to be recorded per appointment.
REQ-22	The system shall maintain a Treatment Catalogue in which every treatment has a unique service code, name, category, price, default duration, and an active/inactive flag, so a treatment can be retired without deleting historical billing records that reference it.
REQ-23	The system shall record, for every treatment administered, the unit price that applied at the time of treatment, independent of any later change to the catalogue price, so that historical invoices are never altered retroactively.

4.5 Billing, Payments, and Insurance Claims

4.5.1 Description and Priority

Covers automatic invoice generation from completed appointments, tracking of full or partial payments and outstanding dues, and submission of insurance claims for reimbursement.

Priority: High

4.5.2 Stimulus/Response Sequences

- An appointment is marked Completed and its treatments are recorded → system automatically generates (or updates) the corresponding invoice with subtotal, insurance-covered amount, and patient-payable amount.
- Patient makes a full or partial payment against an invoice → system updates the amount paid and recalculates the outstanding balance and invoice status.

- Staff member submits an insurance claim for an invoice against a patient's policy → system records the claimed amount and tracks its approval status.

4.5.3 Functional Requirements

ID	Requirement
REQ-24	The system shall automatically generate an invoice once an appointment is marked Completed and its treatments are recorded, without requiring manual invoice creation.
REQ-25	The system shall automatically compute and maintain, for every invoice, the subtotal amount, insurance-covered amount, patient-payable amount, and amount paid; these values shall never be directly editable by an application user, only by the database's own billing logic.
REQ-26	The system shall allow a patient to make one or more full or partial payments against an invoice, using at least the methods Cash, Card, Insurance, and Online, and shall track the total outstanding due at all times.
REQ-27	The system shall classify every invoice as Unpaid, PartiallyPaid, or Paid, based on the amount paid relative to the patient-payable amount, and shall reject any payment that would cause the amount paid to exceed the patient-payable amount.
REQ-28	The system shall allow an insurance claim to be submitted for an invoice against a specific insurance policy, recording the claimed amount, the approved amount, and a status of Pending, Approved, PartiallyApproved, or Rejected.
REQ-29	The system shall calculate the insurance-covered portion of an invoice from the coverage percentage and maximum claimable amount defined on the patient's policy for each treatment (REQ-11), and shall calculate the remaining patient-payable amount accordingly.

4.6 Management Reporting

4.6.1 Description and Priority

Provides the five management reports required by MedSync's clinic management, generated directly from the database so that they always reflect the current state of the data.

Priority: High

4.6.2 Stimulus/Response Sequences

- Branch manager requests the daily appointment summary for their branch → system returns counts of Scheduled, Completed, and Cancelled appointments for the selected date.
- Finance administrator requests the doctor-wise revenue report → system returns gross and collected revenue per doctor.
- Finance administrator requests the list of patients with outstanding balances → system returns every unpaid or partially-paid invoice with the balance due.
- Admin/Finance staff member requests the treatments-per-category report for a date range → system returns treatment counts grouped by category.

- Admin/Finance staff member requests the insurance-vs-out-of-pocket report → system returns, per month, the total insurance-covered amount versus the total patient-paid amount.

4.6.3 Functional Requirements

ID	Requirement
REQ-30	The system shall provide a branch-wise appointment summary per day, showing the number of Scheduled, Completed, and Cancelled appointments for each branch.
REQ-31	The system shall provide a doctor-wise revenue report showing, for each doctor, the gross revenue generated and the revenue actually collected.
REQ-32	The system shall provide a report listing every patient with an outstanding (unpaid or partially paid) balance, together with the amount due.
REQ-33	The system shall provide a report of the number of treatments delivered per treatment category over a management-specified date range.
REQ-34	The system shall provide a report comparing total insurance-covered payments against total out-of-pocket payments, broken down by month.

4.7 QA / Testing User Interface

4.7.1 Description and Priority

A minimal user interface, built only so the QA team can exercise every database rule (booking, overlap rejection, treatment gating, billing, payments, and reports) without writing raw SQL. The UI is explicitly out of scope for production use and carries no functional requirements beyond exposing the features above.

Priority: Medium

4.7.2 Stimulus/Response Sequences

- QA tester logs in with a role (Reception, Clinician, or Admin) → system shows only the screens and actions permitted for that role.
- QA tester attempts an action that violates a business rule (e.g. booking an overlapping slot, recording a treatment on a Scheduled appointment) → the UI surfaces the exact rejection message raised by the database, rather than a generic error.

4.7.3 Functional Requirements

ID	Requirement
REQ-35	The system shall provide a role-based login (Reception, Clinician, Admin) that reflects the corresponding database-level privilege role, and shall expose only the screens and actions permitted for the logged-in role.
REQ-36	The system shall provide screens sufficient to exercise branch/staff management, patient registration and search, appointment booking/reschedule/cancel/walk-in, treatment

ID	Requirement
	recording, invoicing, payments, insurance claims, and all five reports (REQ-1 through REQ-34), and shall display database rejection messages to the user without modification.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-1: The doctor-overlap check shall complete, and either accept or reject a booking request, within 200 ms under a load representative of a single branch's peak booking traffic (approximately 5–10 concurrent booking attempts).
- NFR-2: Each of the five management reports (Section 4.6) shall return results within 2 seconds for a dataset of the scale described in Section 6.4 (up to a few hundred appointments and invoices).
- NFR-3: The system shall support at least 20 concurrent staff users across the three branches without deadlocks; conflicting bookings for the same doctor shall be serialized (one succeeds, the other is rejected) rather than silently corrupting data.

5.2 Safety Requirements

CATMS Phase 1 does not directly control any physical medical equipment, so there are no life-safety hazards from software malfunction in the traditional sense. However, because the system records clinical treatment history and billing, data-integrity failures (a lost treatment record, an incorrect invoice, a silently dropped payment) could cause real harm to patients or the clinic's finances.

Accordingly:

- NFR-4: No committed appointment, treatment, invoice, or payment record shall ever be silently lost; every state-changing operation shall run inside a database transaction that either fully commits or fully rolls back (Section 6.1).
- NFR-5: Every change to an appointment's status and every reschedule shall be captured in an audit trail (Appointment_Status_Log, Appointment_History) that can be reviewed if a dispute arises.

5.3 Security Requirements

- NFR-6: All SQL executed by the application layer shall be parameterized; string-concatenated queries built from user input are prohibited, as the primary defence against SQL injection.
- NFR-7: The database shall define separate roles (e.g. reception, clinician, admin) with GRANT/REVOKE privileges scoped to the tables and views each role needs, so that a compromised API credential cannot access data outside its role's scope (e.g. a reception-scoped credential must never be able to read raw Payment rows).
- NFR-8: Every API route shall independently verify the logged-in user's role before executing a privileged action, as a second, application-level check in addition to the database-level role (defence in depth).
- NFR-9: User passwords/credentials shall never be stored in plain text; they shall be hashed (e.g. with bcrypt) before storage, and credential data shall be kept in a table separate from clinical/patient data.

5.4 Software Quality Attributes

- **Correctness:** every business rule stated in the client brief (no double-booking, no treatment before completion, accurate outstanding balances) is enforced inside the database so it holds regardless of which client calls it.
- **Maintainability:** the schema is normalized to 3NF/BCNF (Section 6.2), with the single deliberate exception documented in Section 6.2, so that updates to one fact (e.g. a treatment's price) do not require updating multiple tables.
- **Reliability:** transactions guarantee atomicity and consistency (Section 6.1) so a failure partway through a multi-step operation (e.g. booking, or recalculating an invoice) cannot leave the database in an inconsistent state.
- **Testability:** a reproducible seed dataset (fixed random seed) allows the same test data to be regenerated for every demonstration or grading run.
- **Usability:** the QA UI visibly disables actions that a business rule would reject, and displays the exact database rejection message, so the rule being tested is transparent to the user.

5.5 Business Rules

- **BR-1:** A doctor may never hold two overlapping, non-cancelled appointments.
- **BR-2:** Treatments and consultation notes may only be recorded against an appointment whose status is Completed.
- **BR-3:** An invoice's subtotal, insurance-covered amount, patient-payable amount, and amount-paid fields are maintained only by the database's own triggers/procedures and are never directly writable by an application user.
- **BR-4:** A payment may never cause an invoice's amount paid to exceed its patient-payable amount.
- **BR-5:** A branch manager is an Employee whose role is Manager; there is no separate manager entity.
- **BR-6:** Only staff with the Admin/Finance role may access billing internals, insurance claim approval, or the five management reports; Reception and Clinician roles are scoped to their respective operational tasks (Section 2.3).

6. Other Requirements

6.1 Database / Transaction Requirements

As explicitly requested by the client brief, the database must guarantee the ACID properties for every state-changing operation:

Property	How CATMS guarantees it
Atomicity	Multi-step operations (e.g. booking an appointment, recalculating an invoice after a new treatment is added) are wrapped in a single stored procedure/transaction that either fully commits or fully rolls back if any step — such as the doctor-overlap check — fails.
Consistency	Primary/foreign keys, CHECK constraints (e.g. appointment end time after start time, coverage percentage between 0 and 100, payment amount greater than zero), and business-rule triggers (overlap, treatment-gating, payment-guard) ensure every committed transaction leaves the database in a valid state.
Isolation	Row-level locking (e.g. SELECT ... FOR UPDATE on a doctor's existing appointments before inserting a new one) prevents two concurrent bookings for the same doctor from both passing the overlap check before either commits.
Durability	Once a transaction commits, the change survives a crash, relying on the underlying DBMS's write-ahead logging; audit tables (Appointment_Status_Log, Appointment_History) additionally make committed history queryable, not just crash-safe.

6.2 Data Modelling and Normalization Requirements

The schema shall model all entities and relationships identified in the client brief, including: Branch, Employee, Doctor (as a specialization of Employee), Specialty, Patient, Emergency_Contact, Insurance_Provider, Insurance_Policy, Policy_Coverage, Appointment, Consultation_Note, Appointment_History, Appointment_Status_Log, Treatment_Catalogue, Appointment_Treatment, Invoice, Payment, and Insurance_Claim.

- The schema shall be normalized to at least 3NF, and preferably BCNF, for every table.
- Any many-valued relationship (e.g. a doctor serving multiple specialties, a policy covering multiple treatments) shall be modelled as a many-to-many junction table, never as a repeating group or comma-separated column.
- The one deliberate denormalization permitted is storing the unit price of a treatment at the time it was administered (Appointment_Treatment.unit_price_at_time), so that a later change to the treatment catalogue's price never silently alters a historical invoice; this exception shall be explicitly documented in the schema design.
- Every table shall have a surrogate primary key (e.g. a SERIAL/auto-incrementing integer) rather than a natural key such as a National Identity Card number, so that primary keys remain stable even if a natural identifier's format changes.

6.3 Indexing Requirements

Indexes shall be created to support the two heaviest access patterns — the doctor-overlap check performed on every booking, and the five management reports — including, at minimum, indexes on Appointment(doctor_id, scheduled_start), Appointment(branch_id, scheduled_start), Invoice(status), and Appointment_Treatment(treatment_id). Low-cardinality columns (e.g. an appointment's status alone) shall not be independently indexed, since a B-tree over 2–3 distinct values adds write overhead without materially improving read performance.

6.4 Dummy Data Requirements

The database shall be populated with realistic dummy data before presentation/grading, sufficient in volume for the five reports to show meaningful patterns rather than trivial results, approximately: 3 branches; ≈20 employees; 10–12 doctors; 8–10 specialties; 60–100 patients (about 40% holding an active insurance policy); 3–4 insurance providers; 15–20 catalogue treatments; 150–300 appointments over a simulated 2–3 month window (a mix of Scheduled, Completed, and Cancelled, approximately 15% walk-ins); and invoices/payments/claims matching every completed appointment, with a mix of fully paid, partially paid, and unpaid invoices. The seed data shall be reproducible (fixed random seed) and re-runnable to reset the database to a known state.

Per the client brief, the QA UI is not required to provide data-entry screens purely for the purpose of loading this dummy data — the seed dataset may be inserted directly with SQL scripts (e.g. a single seed.sql), independently of the UI screens listed in Section 3.1, which exist to demonstrate live use of the system rather than to bulk-load it.

6.5 Internationalisation and Localisation Requirements

Phase 1 targets a single-locale deployment (Sri Lanka); no multi-language or multi-currency support is required. All monetary values shall be expressed in Sri Lankan Rupees (LKR) using DECIMAL (10,2).

6.6 Logical Structure of the Data (Data Dictionary)

This section defines the complete logical structure of the CATMS database: every table, its fields, keys/constraints, and the purpose of each field. It expands Section 6.2 into a full data dictionary and should be read together with Figure 1 (Appendix B), the conceptual module diagram. The physical schema consists of 17 base tables, 2 pure many-to-many junction tables that carry no attributes of their own beyond the relationship (Doctor Specialty, and the composite key of Policy Coverage), and 5 read-only reporting objects (views/functions) 19 stored relations plus reporting layer in total, grouped into the five ownership modules used throughout this document.

Notation used in the Key / Constraint column:

Field	Type	Key / Constraint	Description
note_id	SERIAL	PK	—
appointment_id	INT	FK → Appointment, UNIQUE	Enforces the 1:1 relationship with Appointment.
diagnosis	VARCHAR(200)	—	—
notes	TEXT	—	Free-text consultation notes.

Module A — Branch & Staff Management

Table: Branch

Base table for every staff member (medical and non-medical) at a branch.

Field	Type	Key / Constraint	Description
history_id	SERIAL	PK	—
appointment_id	INT	FK → Appointment	—
old_start / new_start	TIMESTAMP	NOT NULL	The time before and after the reschedule.
changed_by	INT	FK → Employee	Staff member who made the change.
reason	VARCHAR(200)	—	—

Table: Appointment_Status_Log

Audit trail of status transitions (Scheduled → Completed / Cancelled).

Field	Type	Key / Constraint	Description
log_id	SERIAL	PK	—
appointment_id	INT	FK → Appointment	—
old_status / new_status	VARCHAR(10)	—	—
changed_at	TIMESTAMP	DEFAULT now()	Populated automatically by a trigger, never written directly by the application.

Module D — Treatment & Billing

Table: Treatment_Catalogue

Master list of billable services (Consultation, X-Ray, ECG, Injection, etc.).

Field	Type	Key / Constraint	Description
treatment_id	SERIAL	PK	—
service_code	VARCHAR(15)	UNIQUE, NOT NULL	e.g. CONS-GEN, XR-CHEST, ECG-STD.
name	VARCHAR(100)	NOT NULL	—
category	VARCHAR(50)	NOT NULL	Drives the treatments-per-category report.
price	DECIMAL(10,2)	CHECK price >= 0	Current clinic-wide catalogue price.
default_duratio	SMALLINT	DEFAULT 15	—
is_active	BOOLEAN	DEFAULT true	Retires a service without deleting billing history that references it.

Table: Invoice

Auto-computed bill for a completed appointment; 1:1 with Appointment.

Field	Type	Key / Constraint	Description
invoice_id	SERIAL	PK	—
appointment_id	INT	FK → Appointment, UNIQUE	Enforces the 1:1 relationship.
subtotal_amount	DECIMAL(10,2)	DEFAULT 0	Sum of all treatment line items.
insurance_covered_amount	DECIMAL(10,2)	DEFAULT 0	—
patient_payable_amoun	DECIMAL(10,2)	DEFAULT 0	Derived: subtotal – insurance covered; maintained by trigger.

amount_paid	DECIMAL(10,2)	DEFAULT 0	Running total of all payments received.
status	VARCHAR(15)	CHECK IN (Unpaid, PartiallyPaid, Paid)	—

Module E — Payments, Claims & Reporting

Table: Payment

One or more payments (full or partial) settling an invoice.

Field	Type	Key / Constraint	Description
payment_id	SERIAL	PK	—
invoice_id	INT	FK → Invoice, NOT NULL	One invoice can have many payments.
amount	DECIMAL(10,2)	CHECK > 0	Rejected by trigger if it would exceed the outstanding balance.
payment_method	VARCHAR(10)	CHECK IN (Cash, Card, Insurance, Online)	—
received_by	INT	FK → Employee	—
paid_at	TIMESTAMP	DEFAULT now()	—

6.7 Requirements Traceability Matrix

Every key requirement is traced from the business problem/objective that motivates it (Section 1.6/1.7), through its functional or non-functional requirement ID, to the concrete database mechanism that enforces it, and the means by which it will be verified. This closes the loop from problem statement to implementation and test.

Req. ID	Traces to Objective	Enforcing DB Mechanism	Verified By
REQ-01 (Patient registration, Sec. 4.2)	Obj. 1 — Unify patient records	Patient table with UNIQUE nic_or_passport; no branch-scoped partitioning	Cross-branch search test case
REQ-14 (Doctor overlap, Sec. 4.3)	Obj. 2 — Scheduling integrity	Overlap-prevention trigger / EXCLUDE constraint + row-level lock	Concurrent double-booking attempt rejected
REQ-19 (Treatment gating, Sec. 4.4)	Obj. 5 — Enforce rules at data layer	Trigger restricting treatment entry to Completed appointments	Treatment-on-non-Completed-appointment rejected
REQ-25 (Invoice generation, Sec. 4.4/4.5)	Obj. 3 — Digitise treatment-to-invoice pipeline	Invoice auto-recalculation trigger/function	Invoice totals assertion test after treatment entry
REQ-27 (Payment cap, Sec. 4.5)	Obj. 4 — Track billing state precisely	Payment-guard trigger rejecting amounts over the outstanding balance	Overpayment-rejected test case
REQ-30–34 (Five reports, Sec. 4.6)	Obj. 6 — Reporting on demand	Five reporting views / parameterised function (Section 6.6, Reporting Views)	Live report screenshots at demonstration
Section 6.1 (ACID)	Obj. 5 — Enforce rules at data layer	Transaction-wrapped stored procedures (booking, rescheduling, payment posting)	Simulated mid-transaction failure test
Section 6.2 (Normalisation)	Obj. 7 — Ground design in theory	3NF/BCNF-normalised DDL with one documented, justified denormalisation	FD/BCNF proof (companion DB Engineering Specification)

Section 5.3 (Security)	Obj. 5 — Enforce rules at data layer	Parameterised queries end-to-end; role-scoped GRANT/REVOKE database users	Code review; role-access test cases
Section 6.3 (Indexing)	Obj. 8 — Demonstrable QA interface	Composite index on Appointment(doctor_id, scheduled_start), etc.	Query timing against the demo dataset

Appendix A: Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — the four properties a database transaction must guarantee (Section 6.1).
BCNF	Boyce-Codd Normal Form: a stricter version of 3NF requiring that the left-hand side of every non-trivial functional dependency be a superkey.
3NF	Third Normal Form: every non-key attribute depends on the whole primary key and nothing but the primary key.
Primary Key (PK)	A minimal set of attributes that uniquely identifies each row of a table.
Foreign Key (FK)	An attribute (or set of attributes) in one table that references the primary key of another table, enforcing referential integrity.
Trigger	A database procedure that runs automatically in response to an INSERT, UPDATE, or DELETE on a table (e.g. the doctor-overlap check).
Stored Procedure	A named, precompiled sequence of SQL/procedural statements stored in the database and invoked explicitly (e.g. to book an appointment atomically).
View	A virtual table defined by a stored query, used here to expose each of the five management reports without duplicating logic in the application layer.
RBAC	Role-Based Access Control — restricting actions based on a user's assigned role (Reception, Clinician, Admin).
NIC	National Identity Card number, used (together with passport number) as a patient or employee's unique natural identifier.
Walk-in	An appointment created directly by staff at the moment a patient arrives, without a prior booking.
Junction table	A table used to represent a many-to-many relationship between two entities (e.g. Doctor_Specialty, Policy_Coverage, Appointment_Treatment).
Denormalization	A deliberate, documented departure from full normalization for performance or correctness reasons (e.g. storing a treatment's price at billing time).

Appendix B: Analysis Models

The conceptual data model groups the schema into five ownership modules; the high-level entity relationships are summarised below. The full entity-relationship diagram, complete attribute lists, functional-dependency/normalization proof, and DDL are provided in the accompanying CATMS Database Engineering Specification.

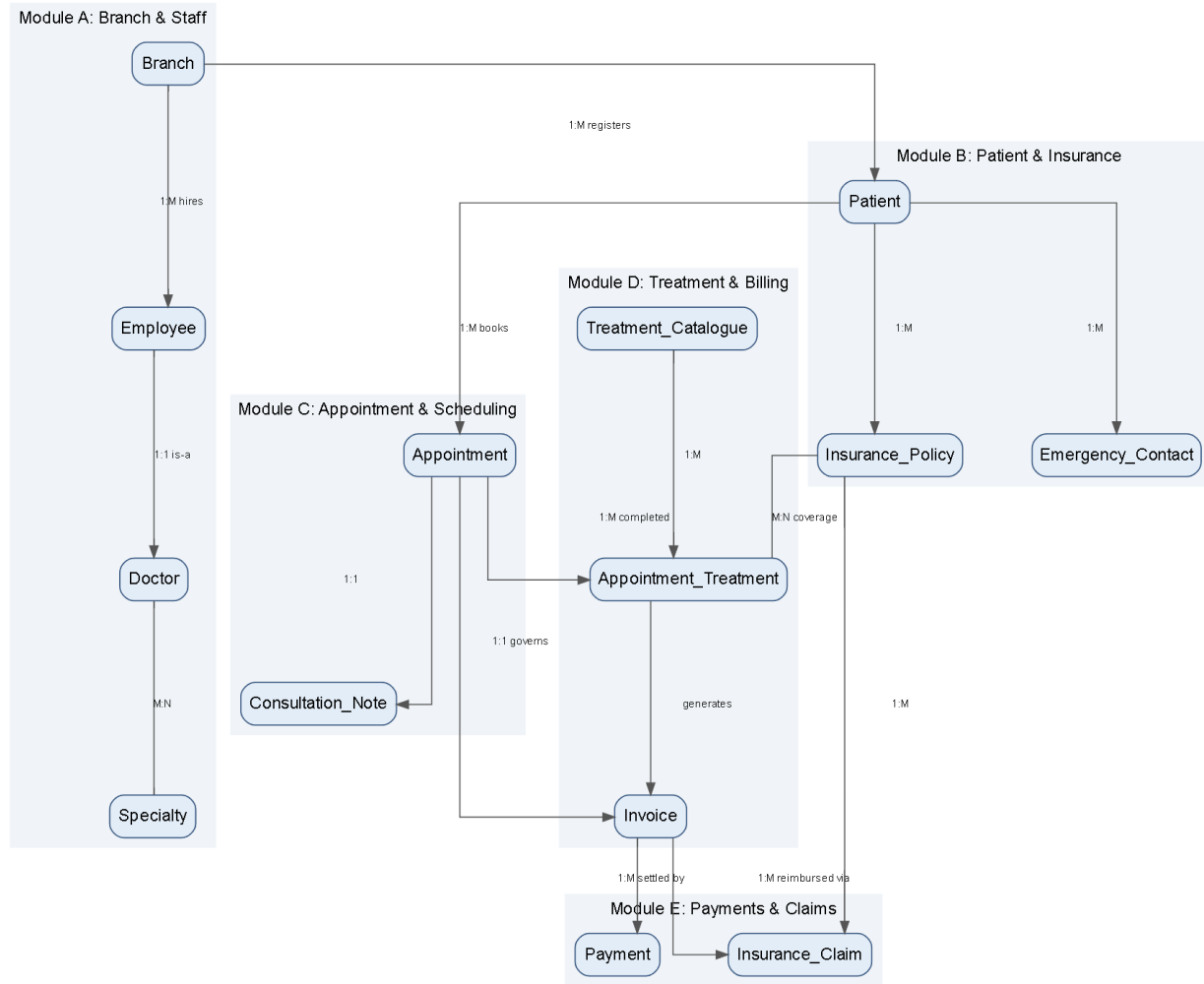
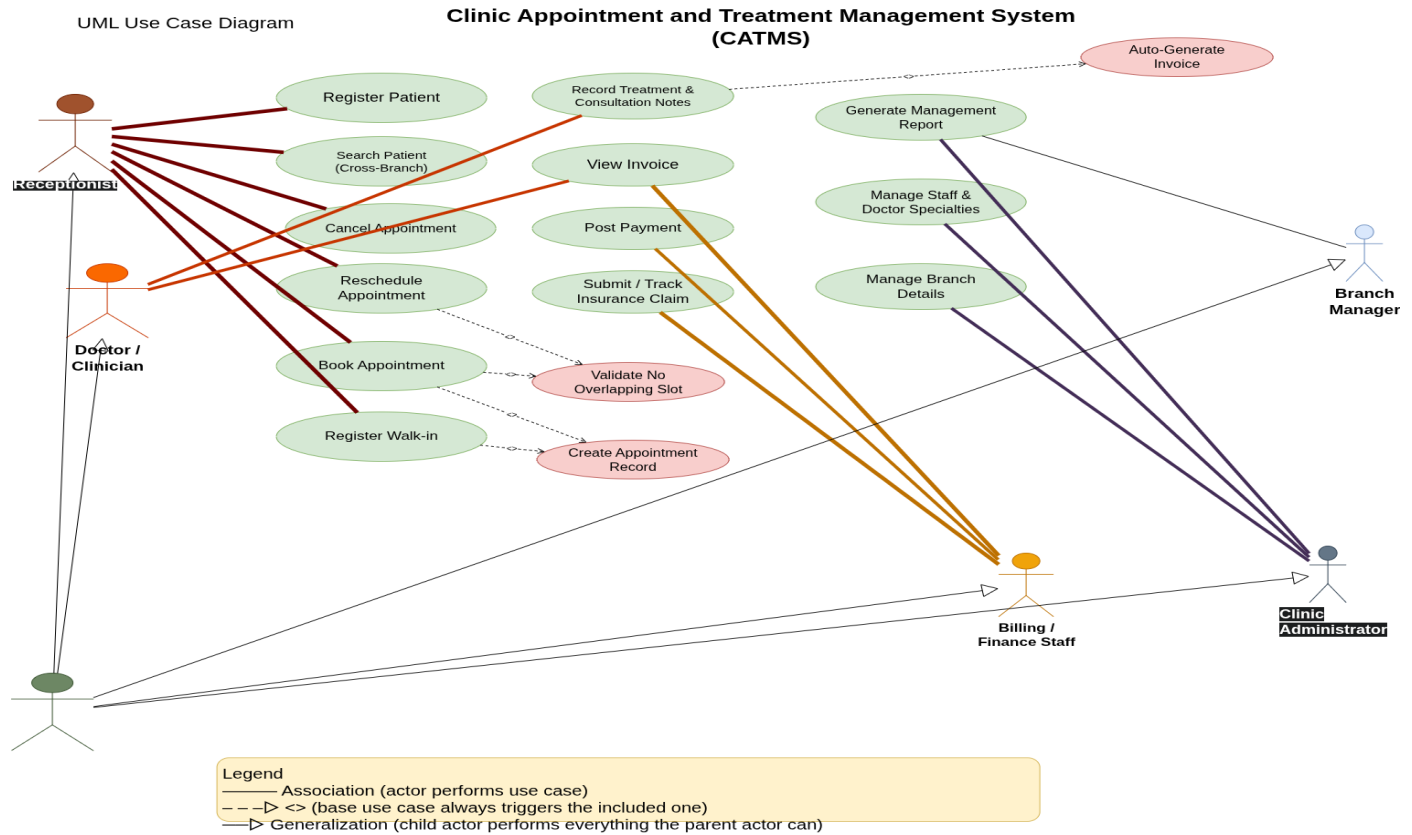


Figure 1: CATMS Conceptual Module and Entity-Relationship Diagram

B.3 Use Case Diagram



B.1 Conceptual Modules

Module	Entities
Branch & Staff	Branch, Employee, Doctor (subtype of Employee), Specialty, Doctor_Specialty
Patient & Insurance	Patient, Emergency_Contact, Insurance_Provider, Insurance_Policy, Policy_Coverage
Appointment & Scheduling	Appointment, Consultation_Note, Appointment_History, Appointment_Status_Log
Treatment & Billing	Treatment_Catalogue, Appointment_Treatment, Invoice
Payments, Claims & Reporting	Payment, Insurance_Claim, and the five reporting views

B.2 Key Relationships and Cardinality

From	To	Cardinality	Meaning
Branch	Employee	1 : M	A branch employs many staff.
Employee	Branch	1 : 1 (manager_id)	A branch has exactly one manager.
Employee	Doctor	1 : 1	Subtype relationship (specialization).
Doctor	Specialty	M : N	Via Doctor_Specialty.
Branch	Patient	1 : M	Registration branch only; record is visible clinic-wide.
Patient	Appointment	1 : M	A patient may have many appointments.
Doctor	Appointment	1 : M	No two overlapping appointments per doctor.
Appointment	Treatment_Catalogue	M : N	Via Appointment_Treatment.
Appointment	Invoice	1 : 1	Created once the appointment is Completed.
Invoice	Payment	1 : M	Supports full or partial settlement.
Invoice	Insurance_Claim	1 : M	Usually one claim, but a bill could split across policies.

Consult the CATMS Database Engineering Specification for the complete Entity-Relationship diagram, the flattened-relation functional-dependency analysis, and the BCNF decomposition proof that produced this schema.

Appendix C: To Be Determined List

#	TBD Item
1	Final list and number of insurance providers and their exact policy terms (currently assumed, not confirmed by MedSync).
2	Whether the target DBMS for grading/deployment is PostgreSQL or MySQL, which affects whether the doctor-overlap rule uses an EXCLUDE constraint or a trigger only.
3	Final choice of hosting environment for the live demonstration (managed cloud Postgres vs. local Docker Compose).
4	Exact authentication mechanism for the QA UI (session cookie vs. JWT) and its expiry policy.
5	Final dummy-data volumes to be used for grading (a range is currently specified in Section 6.4; exact counts to be fixed when the seed script is finalised).
6	Whether branch-specific treatment pricing will be required in a later phase, which would affect the Treatment_Catalogue design.